Laboratorio 4
Tecnologías de Microprocesamiento

Sergio Romero, Anthony Bonino
Ingeniería Mecatrónica – UTEC
Semestre IV
Docentes: Jesús Martínez / Marcelo Díaz

***Resumen*—En este laboratorio se proponen cinco aplicaciones utilizando el microcontrolador ATmega328P programado en lenguaje C. Las actividades integran comunicación RFID, SPI y Bluetooth, lectura de sensores, control de motores y servomotores, visualización mediante LCD y matrices LED RGB, y almacenamiento en EEPROM. El Problema A plantea un cajero automático RFID; el Problema B, una banda transportadora con dos microcontroladores comunicados por SPI; el Problema C, una alarma antirrobo Bluetooth; el Problema D, un juego de memoria con joystick; y el Problema E, un indicador de inclinación con MPU6050. Los sistemas se verificaron mediante simulación en Proteus y montaje físico. Los problemas A, B y D funcionaron en el hardware real; en el Problema C el sistema operó correctamente de forma local, aunque no se logró el enlace Bluetooth con el celular, y en el Problema E el firmware compila sin errores, pero el montaje físico no llegó a funcionar.**

***Palabras clave*—ATmega328P, lenguaje C, RFID, SPI, Bluetooth, PWM, ADC, EEPROM, LDR, DHT11, L298N, WS2812B, MPU6050.**

I. Introducción

El Laboratorio 4 integra diferentes recursos del microcontrolador ATmega328P para resolver problemas de adquisición, control y comunicación. En cada actividad se identifican las entradas y salidas, se implementa el programa en lenguaje C y se verifica el comportamiento mediante simulación y montaje físico. El desarrollo y sus modificaciones se documentan mediante GitHub, junto con las evidencias de funcionamiento que correspondan. Las actividades se desarrollaron según la guía del laboratorio [1] y la hoja de datos del microcontrolador [2].

II. Objetivos

II-A. Objetivo general

Diseñar, programar y verificar aplicaciones embebidas basadas en ATmega328P utilizando lenguaje C y periféricos de comunicación, adquisición, control y almacenamiento.

II-B. Objetivos específicos

- Implementar aplicaciones con RFID, SPI y Bluetooth.
- Adquirir y procesar señales de sensores digitales y analógicos.
- Controlar motores, servomotores, LEDs, buzzer y matrices RGB.
- Implementar máquinas de estados y almacenamiento EEPROM cuando corresponda.
- Registrar pruebas mediante simulación, fotografías y videos.

III. Materiales

- Placas Arduino UNO o Nano con ATmega328P (dos para el Problema B).
- Protoboards, cables de conexión y fuentes de alimentación reguladas adecuadas para los circuitos y motores.
- Lector RFID RC522 y tarjetas MIFARE.
- Teclado matricial 4×4.
- Pantallas LCD 16×2, con adaptador I^2C cuando corresponda.
- Sensores de temperatura DHT11.
- LDR y resistencias de 10 kΩ.
- Potenciómetros y pulsadores.
- Motor DC y módulo puente H L298N.
- Servomotor SG90.
- LEDs de estado y resistencias limitadoras de 220 Ω.
- Buzzers activos y pasivos, transistores NPN (BC547 o 2N2222) y resistencias de 1 kΩ para su accionamiento.
- Resistencias de 330 Ω para la línea de datos de la matriz WS2812B.
- Módulo Bluetooth HC-05 y sensor PIR HC-SR501 o pulsador equivalente.
- Matriz de LEDs RGB 8×8 WS2812B.
- Joystick analógico KY-023 con pulsador.
- Módulo inercial MPU6050.
- Computadora con Microchip Studio, Proteus, Serial Bluetooth Terminal (celular) y GitHub.

IV. Metodología general

Para cada problema se analiza la consigna, se identifican las entradas y salidas, se define la lógica de funcionamiento y se asignan los pines del ATmega328P. Los programas se desarrollan en C y se compilan en Microchip Studio. Los sistemas se verificaron mediante simulación en Proteus y, posteriormente, mediante montaje físico, registrando los resultados, los inconvenientes encontrados y las correcciones realizadas. Las pruebas que no pudieron completarse se indican explícitamente en cada problema. El código fuente y las evidencias se incorporan mediante enlaces al repositorio GitHub y a los archivos compartidos, según su disponibilidad.

V. Problema A: Cajero automático con RFID

V-A. Descripción y funcionamiento

El sistema emula un cajero automático sobre un ATmega328P (Arduino UNO, 16 MHz) programado en C. La au-

tenticación se realiza en dos pasos: una tarjeta MIFARE leída por un módulo RFID RC522 [4] y un PIN de cuatro dígitos ingresado por un teclado matricial 4×4. La interacción con el usuario se realiza mediante una LCD 16×2 con expansor PCF8574 (I^2C, 0x27) [5], [6].

Se administran tres cuentas, cada una con UID de 4 bytes, PIN de cuatro dígitos y saldo de 32 bits (5000, 3000 y 1000 por defecto). El cajero permite consultar el saldo, retirar y depositar; los saldos se conservan en la EEPROM interna. Las operaciones exitosas se señalizan con un destello del LED verde, las fallidas con el LED rojo y el bloqueo con el buzzer. La UART a 9600 baudios se usa como terminal de depuración (UID detectado, PIN correcto o erróneo, bloqueo y monto solicitado).

V-A1. Asignación de pines: La Tabla I resume las conexiones. Las columnas del teclado se distribuyeron entre PD2, PD3, PC3 y PB0 para reservar PD0 y PD1 a la UART.

Cuadro I
ASIGNACIÓN DE PINES DEL PROBLEMA A

Pin Arduino	Puerto AVR	Función
D0	PD0	RX UART (depuración)
D1	PD1	TX UART (depuración)
D2	PD2	Teclado columna 1
D3	PD3	Teclado columna 2
D4	PD4	Teclado fila 1
D5	PD5	Teclado fila 2
D6	PD6	Teclado fila 3
D7	PD7	Teclado fila 4
D8	PB0	Teclado columna 4
D9	PB1	RST (RC522)
D10	PB2	SDA/SS SPI (RC522)
D11	PB3	MOSI SPI (RC522)
D12	PB4	MISO SPI (RC522)
D13	PB5	SCK SPI (RC522)
A0	PC0	LED verde
A1	PC1	LED rojo
A2	PC2	Buzzer
A3	PC3	Teclado columna 3
A4	PC4	SDA I^2C (LCD)
A5	PC5	SCL I^2C (LCD)

Las filas del teclado son salidas y las columnas, entradas con pull-up interna; el barrido pone en bajo una fila a la vez. Los LEDs usan resistencias de 220 Ω y el buzzer se acciona con un transistor 2N2222 con 1 kΩ en la base. El RC522 se alimenta a 3,3 V, ya que no tolera 5 V.

V-B. Lógica de control y transiciones

En espera, el lector se sondea cada 300 ms aproximadamente. Si el UID coincide con una cuenta, se muestra "Bienvenido" con el número de cuenta y se pasa al ingreso del PIN; si no, se indica "Tarjeta no val" y se mantiene la espera. Cada dígito del PIN se muestra como ⋆, la tecla ⋆ borra el último y la validación es automática al cuarto dígito. Al tercer PIN erróneo el sistema muestra "BLOQUEADO", activa el buzzer 1 s, espera 3 s y descarta la sesión.

Desde el menú, A retira, B deposita, C consulta el saldo y D cierra la sesión. El retiro solo se realiza si el monto no supera el saldo ("Saldo insuf." en caso contrario) y el depósito solo si no se desborda el saldo de 32 bits ("Monto invalido"). Los mensajes de resultado se muestran 1,5 s antes de volver al menú.

V-C. Máquina de estados

La lógica se organizó como una máquina de siete estados, descrita en la Tabla II.

Cuadro II
ESTADOS DEL CAJERO AUTOMÁTICO

Estado	Función
`ST_ESPERA_TARJETA`	Sondea el RC522 y compara el UID con las cuentas cargadas en RAM.
`ST_PIN`	Captura 4 dígitos (mostrados como ⋆, ⋆ borra); valida al cuarto dígito; máximo 3 intentos.
`ST_BLOQUEO`	Buzzer durante 1 s y espera de 3 s tras 3 PIN erróneos; reinicia la sesión.
`ST_MENU`	A = retiro, B = depósito, C = saldo, D = salir.
`ST_MONTO_RETIRO`	Captura el monto (⋆ borra, # confirma) y valida el saldo.
`ST_MONTO_DEPOSITO`	Captura el monto y valida el desborde de 32 bits.
`ST_MOSTRAR_SALDO`	Muestra el saldo; cualquier tecla regresa al menú.

Acceder al diagrama de estados (imagen y Draw.io) en GitHub

V-D. Implementación y comunicación

El programa es un bucle principal con un `switch` sobre el estado actual. Una bandera `entrando` indica la primera pasada por cada estado, de modo que el menú se dibuja una sola vez y el resto del tiempo se lee el teclado.

Cuentas y EEPROM. Cada cuenta es una estructura `cuenta_t`. La tabla por defecto (`cuentas_def`) está en flash y se copia a la EEPROM solo cuando el byte de control `ee_magic` no vale `EE_MAGIC` (0xA7), es decir, en el primer arranque o al modificar la tabla; así no hace falta grabar un archivo `.eep`. Tras cada operación se actualiza solo el saldo con `eeprom_update_dword()` [3], que escribe únicamente los bytes que cambiaron.

Tarjeta, teclado y montos. `detectar_tarjeta()` considera válida una lectura cuando el UID no es nulo y lo compara con cada cuenta mediante `memcmp()`. `keypad_scan()` aplica un antirrebote de 25 ms y no bloquea si no hay tecla. `leer_monto()` acepta hasta 6 dígitos, borra con ⋆ y confirma con #.

Comunicación. El ATmega328P es maestro del bus SPI con el RC522; la LCD comparte el bus I^2C en A4/A5, y la UART (`BAUD_PRESCALE` = 103) se usa solo para depuración, con los valores numéricos en hexadecimal.

V-E. *Simulación*

La simulación en Proteus se verificó por etapas: mensajes de la LCD, detección y comparación de UID, captura de PIN con contador de intentos, y operaciones con actualización del saldo en EEPROM. La UART se monitoreó con un terminal virtual a 9600 baudios.

Acceder a la simulación en Proteus

V-F. *Montaje y resultados*

El montaje reúne el RC522, la LCD, el teclado, los LEDs y el buzzer sobre el ATmega328P, con masa común y las conexiones de la Tabla I.

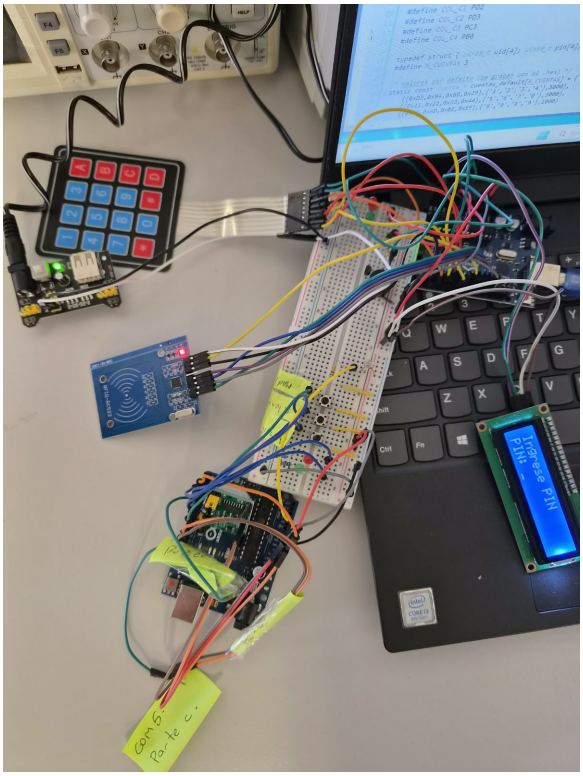


Figura 1. Montaje físico del cajero automático con lector RFID.

Las pruebas se realizaron en simulación y se repitieron en el montaje con la tarjeta registrada en la cuenta 1, con el mismo comportamiento. Se comprobó además que la tabla por defecto solo se copia a la EEPROM en el primer arranque. La Tabla III resume los resultados.

Cuadro III
RESULTADOS DE LAS PRUEBAS DEL PROBLEMA A

Prueba	Resultado
Mensajes en LCD por I^2C	Correcto
Detección de tarjeta y comparación de UID	Correcto
Rechazo de tarjeta no registrada	Correcto
Ingreso de PIN (enmascarado con * y borrado)	Correcto
Bloqueo tras 3 intentos fallidos	Correcto
Retiro con saldo suficiente / insuficiente	Correcto
Depósito y validación de desborde	Correcto
Consulta de saldo	Correcto
Inicialización automática y persistencia en EEPROM	Correcto
Pruebas con montaje físico	Correcto

El cajero integra SPI, TWI/I^2C y UART bajo una máquina de estados; la EEPROM conserva los saldos entre reinicios y las validaciones de PIN, saldo y desborde protegen al sistema frente a operaciones inválidas.

V-G. *Limitaciones*

(i) La lectura del RC522 no valida CRC ni BCC, por lo que el UID no constituye una autenticación segura; (ii) solo la cuenta 1 corresponde a una tarjeta física, y cambiar las tarjetas exige editar `cuentas_def` y `EE_MAGIC`, lo que restablece los saldos; (iii) los PIN se guardan en texto plano; (iv) el contador de intentos está en RAM, por lo que el bloqueo no persiste tras un reinicio; y (v) la LCD y los retardos son bloqueantes, por lo que el teclado no se atiende mientras se muestran mensajes.

V-H. *Video de funcionamiento*

Acceder al video de funcionamiento

V-I. *Repositorio GitHub*

El firmware se desarrolló en C con Microchip Studio, usando la biblioteca del RC522 y rutinas para el teclado, la LCD y la UART. Como `twi_lcd.h` incluye `twi.c`, que también se compila por separado, sus funciones se renombran con `#define` para evitar símbolos duplicados.

Acceder al código fuente

VI. PROBLEMA B: BANDA TRANSPORTADORA CON COMUNICACIÓN SPI

VI-A. *Descripción y funcionamiento*

Se implementó el control de una banda transportadora con dos ATmega328P (Arduino UNO) comunicados por SPI en una arquitectura maestro–esclavo. El maestro adquiere las señales: una LDR detecta el paso de piezas, un DHT11 [8] supervisa la temperatura, un pulsador inicia o detiene el proceso y un potenciómetro fija la velocidad. Una LCD 16×2 por I^2C muestra la temperatura, la velocidad, el estado y el conteo de piezas.

El esclavo recibe las órdenes por SPI y controla los actuadores: el motor DC de la banda mediante un L298N [7]

con PWM, un servomotor SG90 [9] que simula el pistón de expulsión, dos LEDs y un buzzer. Una protección por sobretemperatura detiene el motor y activa una alarma.

VI-A1. Asignación de pines: Las Tablas IV y V presentan las conexiones de cada placa.

Cuadro IV
ASIGNACIÓN DE PINES DEL ARDUINO MAESTRO

Pin	Puerto AVR	Función
A0	PC0	Lectura LDR
A1	PC1	Potenciómetro
D2	PD2	DHT11
D3	PD3	Start/Stop
A4	PC4	SDA LCD
A5	PC5	SCL LCD
D10	PB2	SS SPI
D11	PB3	MOSI SPI
D12	PB4	MISO SPI
D13	PB5	SCK SPI

Cuadro V
ASIGNACIÓN DE PINES DEL ARDUINO ESCLAVO

Pin	Puerto AVR	Función
D2	PD2	LED verde
D3	PD3	LED rojo
D4	PD4	IN1 L298N
D5	PD5	IN2 L298N
D6	PD6	ENA PWM
D7	PD7	Buzzer
D9	PB1	Servo SG90
D10	PB2	SS SPI
D11	PB3	MOSI SPI
D12	PB4	MISO SPI
D13	PB5	SCK SPI

La LDR forma un divisor con una resistencia de 10 kΩ a GND, el potenciómetro se conecta entre 5 V y GND con el cursor en A1, y el pulsador Start/Stop usa la pull-up interna. En el esclavo, IN1 e IN2 fijan el sentido del motor y ENA (D6) recibe el PWM del Timer0; el LED verde indica marcha, el rojo indica alarma y el buzzer se acciona con un BC547 con 1 kΩ en la base.

Acceder al diagrama de estados (imagen y Draw.io) en GitHub

VI-B. Protocolo SPI y lógica de control

Arranque y detención. Ante una pulsación válida, el maestro cambia el estado de marcha y envía la orden al esclavo, que habilita o deshabilita el motor.

Regulación de velocidad. El potenciómetro se lee con el ADC de 10 bits y se muestra en la LCD como `V` (0–255). Como en las pruebas el motor solo giraba con PWM entre 189 y 254, se ajustó por software el valor enviado al esclavo:

$$PWM = \begin{cases} 0, & V = 0, \\ 189 + \left\lfloor \dfrac{65(V-1)}{254} \right\rfloor, & 1 \le V \le 255. \end{cases} \quad (1)$$

Así se aprovecha todo el recorrido del potenciómetro sin cambiar el circuito.

Detección de piezas. La tensión de la LDR se compara con un umbral; cada detección válida incrementa el contador de la LCD y ordena al esclavo accionar el servo, que simula la expulsión y vuelve a reposo.

Protección por temperatura. Con una lectura de 28 °C o más, el maestro detiene el proceso y envía la alarma: el esclavo apaga el motor, enciende el LED rojo y el buzzer y lleva el servo a reposo. La LCD muestra `EMERGENCIA T:28C` y `MOTOR DETENIDO`. La alarma se desactiva a 26 °C o menos (histéresis de 2 °C), pero la banda sigue detenida hasta una nueva pulsación de Start. Los umbrales se eligieron para facilitar la demostración, ya que la consigna no fija uno.

VI-C. Implementación y comunicación

Las placas se conectaron por las cuatro líneas SPI y GND común (Tabla VI). El maestro genera el reloj SCK y selecciona al esclavo con SS.

Cuadro VI
CONEXIONES SPI ENTRE MAESTRO Y ESCLAVO

Señal	Maestro	Esclavo
SS	D10	D10
MOSI	D11	D11
MISO	D12	D12
SCK	D13	D13
GND	GND	GND

Los datos se intercambian en tramas de dos bytes, `[COMANDO] [VALOR]`: el primero identifica la acción (arranque, detención, velocidad PWM, pistón o emergencia) y el segundo, su parámetro.

VI-D. Montaje y resultados

El maestro se conectó a los sensores, el pulsador, el potenciómetro y la LCD; el esclavo, al L298N, el motor, el servo, los LEDs y el buzzer, con GND común entre las placas.

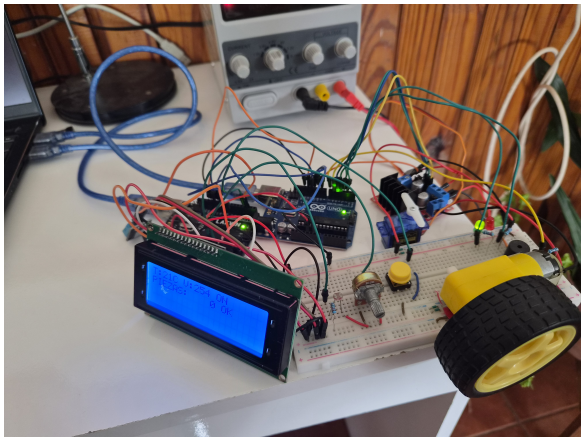


Figura 2. Montaje físico del sistema maestro–esclavo para el control de una banda transportadora.

Se verificaron el arranque con Start/Stop, la regulación de velocidad con el PWM ajustado, la detección de piezas al cubrir la LDR con el accionamiento del servo, y la protección por temperatura. Durante las pruebas surgieron dos inconvenientes: la detección de piezas fallaba porque el divisor de la LDR tenía 1 kΩ en lugar de 10 kΩ, y el primer DHT11 no entregaba lecturas, lo que se resolvió revisando sus terminales y reemplazándolo. Queda por registrar como evidencia la secuencia completa de activación y rearme de la alarma.

Cuadro VII
RESULTADOS DE LAS PRUEBAS DEL PROBLEMA B

Prueba	Resultado
Visualización de datos en LCD	Correcto
Pulsador Start/Stop	Correcto
Lectura del potenciómetro	Correcto
Regulación PWM ajustada (189–254)	Correcto
Comunicación SPI para motor y servo	Correcto
Detección de piezas mediante LDR	Correcto
Accionamiento del servo SG90	Correcto
Lectura del sensor DHT11	Funcional
Umbrales de alarma: 28/26 °C	Implementados
Mensaje de emergencia y temperatura	Implementado
Rearme completo LED rojo/buzzer	Correcto

Se logró un sistema distribuido con dos ATmega328P que controla el arranque, la velocidad y la detección de piezas mediante SPI, integrando entradas analógicas, PWM y actuadores en una aplicación de automatización.

VI-E. Simulación

La simulación permite verificar la comunicación SPI y las señales de control de los actuadores. Los archivos se encuentran en el repositorio.

Acceder a la simulación en Proteus

VI-F. Video de funcionamiento

Acceder al video de funcionamiento

VI-G. Repositorio GitHub

El firmware se organizó en dos proyectos de Microchip Studio, maestro y esclavo. En el maestro se integraron las bibliotecas TWI y LCD (PCF8574, 0x27), ajustando la inicialización del TWI para preservar A0 y A1 y agregando esperas acotadas en las transacciones I^2C. El maestro incorpora además el mapeo de PWM y la alarma con histéresis.

Acceder al código C del maestro

Acceder al código C del esclavo

VII. PROBLEMA C: ALARMA ANTIRROBO CON BLUETOOTH

VII-A. Descripción y funcionamiento

El sistema emula una alarma antirrobo con supervisión inalámbrica sobre un ATmega328P. El microcontrolador es el *nodo planta*: lee los sensores, controla la sirena y reporta su estado por Bluetooth mediante un HC-05 [10]. El *nodo remoto* previsto es un celular con Serial Bluetooth Terminal, que recibe notificaciones y envía comandos.

Una LDR simula la apertura de una ventana (zona `VENTANA`) y un PIR HC-SR501 [11] la detección de movimiento (zona `MOVIMIENTO`). Un LED verde indica sistema armado, un LED rojo indica alarma y un buzzer activo hace de sirena. Una LCD por I^2C muestra el estado, el modo, la zona y el último comando.

En modo **AUTO** una intrusión con el sistema armado dispara la sirena; en modo **MANUAL** solo se notifica y el operador decide desde el celular. Tres pulsadores locales permiten armar o desarmar, silenciar y cambiar el modo.

Aclaración: en el montaje no se logró la comunicación entre el celular y el HC-05, por lo que el control remoto no pudo verificarse. El resto del sistema funcionó de forma local (Sección VII-C1).

VII-A1. Asignación de pines: La Tabla VIII resume las conexiones. El HC-05 usa la UART por hardware.

Cuadro VIII
ASIGNACIÓN DE PINES DEL PROBLEMA C

Pin Arduino	Puerto AVR	Función
D0	PD0	RXD ← TX del HC-05
D1	PD1	TXD → RX del HC-05 (con divisor)
D2	PD2	Sensor PIR / pulsador de movimiento
D3	PD3	LED verde (armado)
D4	PD4	LED rojo (alarma)
D5	PD5	Buzzer activo (sirena)
D6	PD6	Pulsador ARMAR/DESARMAR
D7	PD7	Pulsador SILENCIAR
D8	PB0	Pulsador AUTO/MANUAL
A2	PC2	LDR (ADC2)
A4	PC4	SDA I^2C (LCD)
A5	PC5	SCL I^2C (LCD)

La entrada RX del HC-05 trabaja a 3,3 V, por lo que la línea TXD del microcontrolador pasa por un divisor de tensión. La LDR forma un divisor con 10 kΩ y se lee en A2, porque `twi_init()` configura PC0 y PC1 como salidas. Los pulsadores usan pull-up interna, los LEDs resistencias de 220 Ω y el buzzer se acciona con un 2N2222 con 1 kΩ en la base.

VII-B. Máquina de estados y modos de operación

La lógica tiene cuatro estados (Tabla IX). El modo AUTO/MANUAL es una variable aparte que solo cambia el efecto de una intrusión.

Cuadro IX
ESTADOS DE LA ALARMA ANTIRROBO

Estado	Función
ST_DESARMADO	Estado inicial. Los sensores no se evalúan. LEDs y buzzer apagados.
ST_ARMADO	Vigilancia activa: se evalúan la LDR y el PIR. LED verde encendido.
ST_ALARMA	Intrusión confirmada o alarma forzada. Buzzer encendido y LED rojo intermitente (250 ms); LED verde apagado.
ST_SILENCIADA	Sirena apagada sin desarmar: LED verde y LED rojo encendidos de forma fija, indicando que la alarma quedó memorizada.

ARMAR pasa de DESARMADO o SILENCIADA a ARMADO; una intrusión en modo AUTO lleva a ALARMA, y en modo MANUAL solo envía EVENTO:DETECTADO. FORZAR dispara la alarma de forma remota, SILENCIAR pasa a SILENCIADA y DESARMAR vuelve a DESARMADO desde cualquier estado. Como los comandos remotos y los pulsadores pasan por la misma función (`handle_command()`), todas las transiciones con pulsador pudieron verificarse sin Bluetooth; FORZAR no tiene pulsador y no pudo probarse.

Acceder al diagrama de estados del Problema C (imagen y Draw.io) en GitHub

VII-C. Implementación y comunicación

El lazo principal no usa retardos bloqueantes: el Timer0 en modo CTC genera una base de tiempo de 1 ms con la que se planifican la lectura de sensores y pulsadores (20 ms), el parpadeo del LED rojo (250 ms), la LCD (como máximo cada 100 ms) y el reporte de estado (5 s). Un evento de intrusión se valida tras tres lecturas consecutivas activas (60 ms), y la LDR se compara con un umbral de 600 cuentas.

VII-C1. Comunicación Bluetooth: El HC-05 funciona como puente transparente con la UART (9600 baudios, 8N1). La recepción se hace por la interrupción `USART_RX_vect` en un buffer circular de 64 bytes, de modo que no se pierden caracteres. Los comandos de la Tabla X se aceptan en mayúsculas o minúsculas y se confirman con ACK:<comando> (ERR:COMANDO si no son válidos).

Cuadro X
COMANDOS ACEPTADOS POR EL NODO PLANTA

Comando	Acción
ARMAR	Arma el sistema desde DESARMADO o SILENCIADA.
DESARMAR	Desarma el sistema desde cualquier estado.
SILENCIAR	Apaga la sirena sin desarmar (solo en ALARMA).
AUTO	Selecciona el modo automático.
MANUAL	Selecciona el modo manual.
FORZAR	Dispara la alarma de forma remota (zona REMOTO).
ESTADO	Solicita el estado actual del sistema.

El estado se reporta con una trama CLAVE:VALOR en cada cambio y cada 5 s, por ejemplo:

```
ALARMA:ON;ZONA:VENTANA;
ESTADO:ALARMA;MODO:AUTO
```

Estado de la comunicación. El enlace entre el celular y el HC-05 no pudo hacerse funcionar. Entre las posibles causas, no confirmadas, están el conversor USB del Arduino conectado a la misma línea PD0, una velocidad del módulo distinta de 9600 baudios (se verifica en modo AT con AT+UART?) o problemas de emparejamiento en la aplicación. Como mejora se propone conectar el HC-05 a un puerto serie por software y dejar la UART por hardware para depuración.

VII-D. Montaje y resultados

El montaje reúne el HC-05, la LDR, el PIR, los LEDs, el buzzer, los pulsadores y la LCD (Tabla VIII). El HC-05 debe desconectarse al cargar el programa, porque comparte PD0/PD1 con el USB.

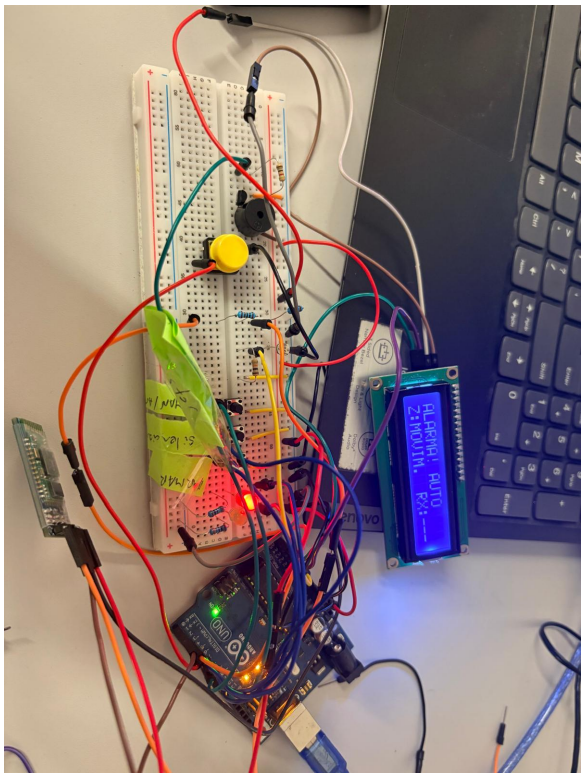


Figura 3. Montaje de la alarma con HC-05.

Al no lograrse el enlace Bluetooth, las pruebas se hicieron de forma local. Se verificaron el armado y desarmado con pulsador, el disparo en modo AUTO al iluminar la LDR y al activar el sensor de movimiento, el modo MANUAL sin disparo de sirena y el silenciamiento sin desarmar.

Cuadro XI
RESULTADOS DE LAS PRUEBAS DEL PROBLEMA C

Prueba	Resultado
Enlace Bluetooth y reporte periódico en el celular	No logrado
Respuesta `ACK` / `ERR` a comandos remotos	No logrado
Armado y desarmado con pulsador local	Correcto
Armado y desarmado remoto	No logrado
Detección por LDR (zona `VENTANA`)	Correcto
Detección por sensor de movimiento (zona `MOVIMIENTO`)	Correcto
Disparo automático en modo AUTO	Correcto
Modo MANUAL sin disparo de sirena	Correcto
Notificación `EVENTO:DETECTADO` en el celular	No logrado
Alarma forzada (`FORZAR`)	No logrado
Silenciamiento sin desarmar (pulsador)	Correcto
Visualización en LCD	Correcto

El funcionamiento local fue correcto, pero la supervisión inalámbrica no pudo verificarse y queda pendiente resolver el enlace con el HC-05.

VII-E. Monitoreo de variables

No se dispone de capturas de Serial Bluetooth Terminal, ya que no se logró la comunicación entre el celular y el HC-05.

VII-F. Simulación

En Proteus, el HC-05 se conectó a PD0/PD1 y la salida PD1 se monitoreó con un terminal virtual a 9600 baudios, conectado solo por su entrada RXD. Al iniciar, el sistema envió `SISTEMA:ALARMA LISTO` y luego el reporte `ALARMA:OFF;ZONA:NINGUNA; ESTADO:DESARMADO;MODO:AUTO` cada 5 s, mientras la LCD mostraba `DESARM. AUTO` y `Z:NINGUNA RX:--`. Esto confirma que el microcontrolador genera las tramas, por lo que la falla del montaje se ubica en el enlace Bluetooth.

Acceder a la simulación en Proteus

VII-G. Limitaciones

(i) No se logró la comunicación Bluetooth, por lo que el sistema solo se operó de forma local; (ii) el HC-05 comparte la UART con el USB, lo que impide depurar y usar Bluetooth a la vez; (iii) el umbral de la LDR depende de la iluminación ambiente; y (iv) los comandos viajan en texto plano y sin autenticación.

VII-H. Video de funcionamiento

El video muestra el funcionamiento local del sistema con los pulsadores.

Acceder al video de funcionamiento

VII-I. Repositorio GitHub

El firmware usa las bibliotecas de los docentes `UART` y `twi_lcd.h`; como esta última incluye `twi.c`, ese archivo no se agrega al proyecto para evitar definiciones duplicadas.

Acceder al código en GitHub

VIII. PROBLEMA D: JUEGO DE MEMORIA CON JOYSTICK Y MATRIZ RGB

VIII-A. Descripción y funcionamiento

Se implementó el juego “Simón Dice” sobre un ATmega328P con una matriz RGB 8×8 WS2812B [12], un joystick KY-023 con pulsador y un buzzer pasivo. El juego genera una secuencia aleatoria creciente de posiciones y colores, la reproduce con luz y sonido, y el jugador debe repetirla ubicando el cursor en cada posición y confirmando con el pulsador. Cada color tiene un tono fijo. El récord se guarda en EEPROM y la UART a 9600 baudios informa el nivel, el color y el fin de la partida.

VIII-A1. Asignación de pines: La Tabla XII resume las conexiones.

Cuadro XII
ASIGNACIÓN DE PINES DEL PROBLEMA D

Pin Arduino	Puerto AVR	Función
D7	PD7	DIN de la matriz WS2812B
A0	PC0	Joystick VRx (ADC)
A1	PC1	Joystick VRy (ADC)
D2	PD2	Pulsador del joystick
D11	PB3	Buzzer pasivo (OC2A)
D4	PD4	Pulsador de reinicio (PCINT20)
D1	PD1	TX UART (depuración)
A2, A3	PC2, PC3	Canales flotantes (semilla aleatoria)

La matriz se alimenta con una fuente externa de 5 V con GND común y una resistencia de 330 Ω en serie con DIN. Los pulsadores usan pull-up interna.

VIII-B. Máquina de estados

El firmware tiene seis estados (Tabla XIII). El pulsador de D4 reinicia la partida desde cualquier estado conservando el récord. La secuencia no se regenera en cada nivel: se conserva y se agrega un paso, como en el Simón Dice comercial.

Cuadro XIII
ESTADOS DEL JUEGO DE MEMORIA

Estado	Función
`ST_LIBRE`	Reposo. El cursor se mueve libremente; un pulso corto cambia el color de forma aleatoria (con tono de 90 ms) y un pulso largo ($\geq$ 0,9 s) inicia la partida en el nivel 1 con una secuencia de 3 pasos.
`ST_SHOW`	Reproduce la secuencia: cada posición se enciende 500 ms con 200 ms de apagado, con el tono de su color.
`ST_PLAY`	El jugador mueve el cursor y confirma con el pulsador (corto o largo). Si la posición coincide con el paso actual, suena el tono de ese paso y avanza; los pasos acertados quedan encendidos con brillo tenue. Si no coincide, la partida termina.
`ST_OK`	Animación de acierto (dos destellos verdes y arpegio de 523, 659 y 784 Hz). Actualiza el récord en EEPROM si el nivel lo supera. Si la secuencia alcanzó 16 pasos, ejecuta la animación de victoria y vuelve a `ST_LIBRE`; si no, agrega un paso y pasa a `ST_PAUSA`.
`ST_PAUSA`	Modo libre entre niveles: se puede mover el cursor y cambiar el color; un pulso largo inicia el siguiente nivel.
`ST_ERROR`	Animación de fallo (tres destellos rojos y tono de 150 Hz durante 600 ms). Informa el nivel y el récord por UART y regresa a `ST_LIBRE` en el nivel 1, con el cursor en (3,3) y color rojo.

Acceder al diagrama de estados (imagen y Draw.io) en GitHub

VIII-C. Implementación y comunicación

Matriz WS2812B. Cada bit dura ≈1,25 μs con tolerancia mínima, por lo que la transmisión se implementó por *bit-banging* con ensamblador en línea de ciclos fijos. Cada cuadro envía 64 LEDs $\times$ 24 bits (GRB) con las interrupciones deshabilitadas, seguido de un *latch* de 300 μs.

Sonido. El buzzer se maneja con el Timer2 en modo CTC con conmutación de OC2A y prescaler 256:

$$f = \frac{F_{CPU}}{2\,N\,(\mathrm{OCR2A}+1)} \tag{2}$$

Las frecuencias son 392 Hz (rojo), 523 Hz (verde), 262 Hz (azul) y 659 Hz (amarillo).

Joystick. Al arrancar se calibra el centro promediando 16 lecturas por eje, y la zona muerta es el 30 % del centro (mínimo 10 cuentas), lo que permite usar el mismo programa en Proteus y en la placa. El cursor se mueve en un eje por vez, cada 160 ms como mínimo, dentro del rango $[0, 7]$.

Aleatoriedad y EEPROM. `rand()` se siembra con lecturas de canales ADC flotantes. El récord se guarda con `eeprom_update_*` y el valor 0xFFFF (EEPROM virgen) se toma como récord cero.

Simulación y placa. La constante `SIMULACION` selecciona la referencia AREF y brillo 255 en Proteus, o AV_{CC} y brillo 60 en la placa real para reducir el consumo.

VIII-D. Montaje y resultados

El montaje reúne la matriz con fuente externa, el joystick, el buzzer y el pulsador de reinicio, con las conexiones de la Tabla XII.

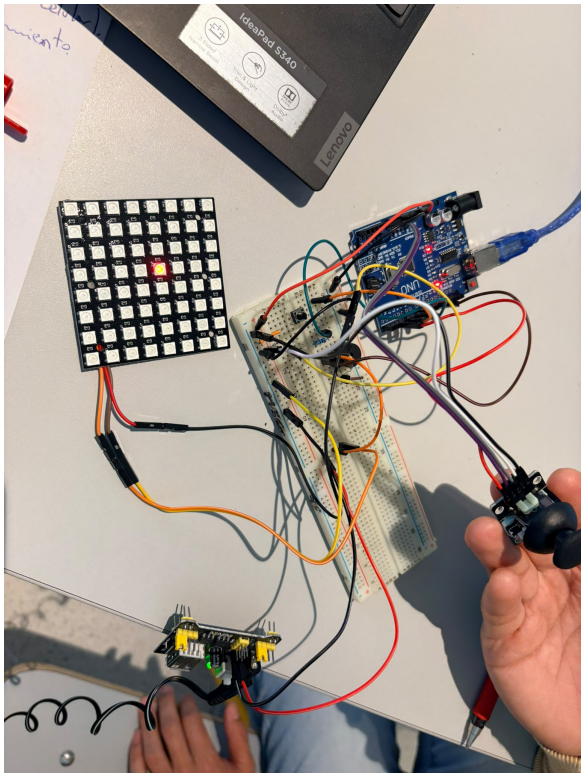


Figura 4. Montaje del juego de memoria en matriz RGB.

En el hardware se verificaron la generación aleatoria de secuencias, la zona muerta del joystick, los tiempos de reproducción, el guardado del récord tras un corte de alimentación, el reinicio por D4 y los mensajes por UART (Tabla XIV).

Cuadro XIV
RESULTADOS DE LAS PRUEBAS DEL PROBLEMA D

Prueba	Resultado
Integridad de la señal DIN (simulación)	Correcto
Secuencias aleatorias distintas entre encendidos	Correcto
Zona muerta y límites del cursor	Correcto
Reproducción de secuencia (500/200 ms)	Correcto
Tonos asociados a cada color	Correcto
Acierto por posición	Correcto
Fin de partida y guardado del récord en EEPROM	Correcto
Reinicio físico mediante D4	Correcto
Mensajes de estado por UART	Correcto

El juego integra ADC, Timer2, UART, EEPROM e interrupciones por cambio de pin; el control de la matriz resultó estable y el récord se conserva entre reinicios.

VIII-E. Simulación

En Proteus se verificó con el osciloscopio virtual la señal DIN sobre D7 (ráfagas de ∼1,9 ms con bits de ≈1,25 μs). En el terminal virtual se observó una partida completa: cambios de color en modo libre, `Inicia nivel: 1`, `Tu turno: repeti las posiciones` y, ante un error, `Error en nivel: 1`, `Record: 0` y `Fin de partida; vuelve al nivel 1`.

Acceder a la simulación en Proteus

VIII-F. Limitaciones

(i) La aleatoriedad proviene de canales ADC flotantes y no es criptográfica; (ii) durante cada transmisión a la matriz (∼1,9 ms) las interrupciones están deshabilitadas; y (iii) las animaciones y la espera del pulsador usan retardos bloqueantes, por lo que el reinicio por D4 se procesa recién al terminar la rutina en curso.

VIII-G. Video de funcionamiento

Acceder al video de funcionamiento

VIII-H. Repositorio GitHub

El programa se desarrolló en C con Microchip Studio, organizado alrededor de la máquina de estados, con funciones para la matriz, el joystick, el sonido y la EEPROM.

Acceder al código fuente

IX. Problema E: Burbuja de inclinación con MPU6050

IX-A. Descripción y funcionamiento

El sistema implementa un nivel de burbuja electrónico sobre un ATmega328P. Un MPU6050 [13] conectado por I^{2}C mide la inclinación y una matriz WS2812B muestra una burbuja que se desplaza en el mismo sentido: el *roll* controla la columna y el *pitch*, la fila.

El color indica el eje predominante: rojo para X, verde para Y, amarillo para inclinación combinada y azul cuando está nivelado (ambos ángulos dentro de ±3°). Los ángulos de ±30° corresponden a los bordes; más allá, la burbuja queda en el borde con un pitido y parpadeo. El pulsador HOLD congela la burbuja y CALIBRAR toma la posición actual como cero y la guarda en EEPROM. Una LCD por I^{2}C y la UART a 38400 baudios muestran los ángulos y el modo.

IX-A1. Asignación de pines: La Tabla XV resume las conexiones. El MPU6050 (0x68) y la LCD (0x27) comparten el bus I^{2}C.

Cuadro XV
Asignación de pines del Problema E

Pin Arduino	Puerto AVR	Función
D0	PD0	RX UART (monitor serial)
D1	PD1	TX UART (monitor serial)
D2	PD2	Pulsador HOLD
D3	PD3	Pulsador CALIBRAR
D6	PD6	DIN de la matriz WS2812B
D8	PB0	Buzzer activo
A4	PC4	SDA I^{2}C (MPU6050 y LCD)
A5	PC5	SCL I^{2}C (MPU6050 y LCD)

El MPU6050 se alimenta a 5 V (regulador incorporado) con AD0 a GND. La matriz usa fuente externa de 5 V, 330 Ω en DIN y un capacitor en la alimentación. Los pulsadores usan pull-up interna.

IX-B. Tratamiento de señales

El acelerómetro se configuró en ±2 g, el giroscopio en ±250 °/s y el filtro pasa bajos interno en ∼44 Hz. Los ángulos se calculan a partir de las aceleraciones:

$$\phi_{acc} = \operatorname{atan2}(a_y,\, a_z) \tag{3}$$

$$\theta_{acc} = \operatorname{atan2}\Big(-a_x,\, \sqrt{a_y^2 + a_z^2}\Big) \tag{4}$$

Como el acelerómetro es ruidoso y el giroscopio deriva, se combinan con un filtro complementario:

$$\phi_k = \alpha\,(\phi_{k-1} + \omega_x\,\Delta t) + (1-\alpha)\,\phi_{acc} \tag{5}$$

con $\alpha = 0{,}98$ y Δt medido en cada ciclo con el Timer1. A la salida se aplica una media móvil exponencial ($\beta = 0{,}70$), se resta el offset de calibración y se aplica la zona muerta de ±3°. Cada ángulo se mapea a una coordenada de la matriz dividiendo $[-30°, 30°]$ en ocho franjas de 7,5°:

$$c = \min\Big(7,\, \Big\lfloor 8\,\frac{\phi + 30}{60} \Big\rfloor\Big) \tag{6}$$

Un eje se considera predominante cuando su ángulo supera 1,5 veces al del otro.

IX-C. Máquina de estados

La lógica tiene cuatro estados (Tabla XVI). HOLD alterna entre `MEDICION` y `HOLD`, y CALIBRAR lleva a `CALIBRACION` desde ambos; al salir, los filtros se reinician con la lectura actual.

Cuadro XVI
ESTADOS DEL INDICADOR DE INCLINACIÓN

Estado	Función
`INICIO`	Configura los periféricos, verifica el MPU6050 y lee la EEPROM. Si hay offsets válidos pasa a `MEDICION`; si no, a `CALIBRACION`.
`CALIBRACION`	Muestra cuatro LEDs blancos en el centro, promedia 64 muestras en reposo, guarda los offsets en EEPROM, emite un pitido de confirmación y pasa a `MEDICION`.
`MEDICION`	Lee el sensor cada 20 ms, filtra, mapea y dibuja la burbuja con su color. Si la burbuja llega a un borde, pita una vez y el LED parpadea.
`HOLD`	No lee el sensor: la burbuja conserva posición y color, y la LCD indica `HOLD`. Al volver a pulsar HOLD regresa a `MEDICION`.

Acceder al diagrama de estados (imagen y Draw.io) en GitHub

IX-D. Implementación y comunicación

El lazo tiene un período fijo de 20 ms medido con el Timer1, y la LCD y la UART se actualizan en ciclos alternados cada 200 ms para no exceder ese período. La matriz se maneja por *bit-banging* como en el Problema D, teniendo en cuenta que es serpenteante (las filas impares se recorren al revés). El buzzer y el aviso de borde son no bloqueantes.

La calibración promedia 64 muestras y guarda los offsets y el bias del giroscopio en una estructura `EEMEM` con un byte de validez (0xA7); si los datos leídos no son válidos, el sistema recalibra.

El MPU6050 se lee en una ráfaga de 14 bytes desde el registro 0x3B con *repeated start*. Como `twi_write_cmd()` de la biblioteca de los docentes espera el código 0x18, válido solo para escritura, se agregó `twi_write_sla_r()`, que espera el código 0x40 de la dirección de lectura. Al arrancar se imprime `WHO_AM_I`, que debería valer 0x68.

IX-E. Montaje y resultados

Se armó el montaje con el MPU6050 y la LCD en el bus I^2C, la matriz con fuente externa, los pulsadores y el buzzer, según la Tabla XV.

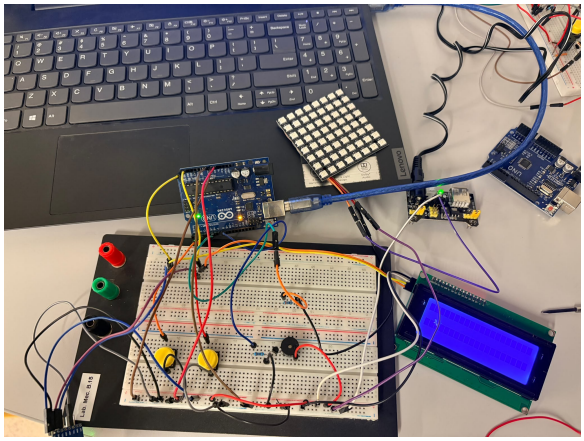


Figura 5. Montaje del indicador de inclinación con MPU6050.

Sin embargo, no se logró que el montaje funcionara dentro del plazo del laboratorio, por lo que las pruebas sobre el hardware no se completaron y no se registró un video. Quedan pendientes la lectura de `WHO_AM_I`, el centrado en reposo, el desplazamiento y color por eje, la saturación en el borde, el modo HOLD y la persistencia de la calibración.

Cuadro XVII
RESULTADOS DE LAS PRUEBAS DEL PROBLEMA E

Prueba	Resultado
Compilación sin errores ni advertencias	Correcto
Montaje físico funcional	No logrado
Comunicación I^2C (`WHO_AM_I` = 0x68)	No verificado
Burbuja centrada y azul en reposo	No verificado
Desplazamiento y color por eje	No verificado
Saturación en los bordes (>30°)	No verificado
Pitido y parpadeo en el límite	No verificado
Modo HOLD	No verificado
Calibración y persistencia en EEPROM	No verificado
Ángulos en LCD y monitor serial	No verificado

El firmware compila sin errores, pero sus características (filtrado, zona muerta, calibración persistente) quedan sin verificación experimental.

IX-F. Simulación

La simulación se planteó por etapas: comunicación I^2C y lectura de `WHO_AM_I`, señal DIN de la matriz, mensajes en la LCD y en el terminal a 38400 baudios, y respuesta a los pulsadores.

Acceder a la simulación en Proteus y capturas del Problema E

IX-G. Limitaciones

(i) La biblioteca TWI espera los códigos de estado sin límite de tiempo, por lo que un fallo en el bus bloquea el programa; (ii) el acelerómetro no mide el giro sobre el eje vertical, que no es necesario aquí; (iii) la matriz no tiene un LED central exacto, por lo que la posición nivelada es la celda (4, 4); y (iv) el sistema no pudo validarse sobre el hardware real.

IX-H. Repositorio GitHub

En el proyecto solo se compila `main.c`, que incluye las bibliotecas de los docentes, y se enlaza la biblioteca matemática. El programa ocupa unos 11,9 kB de flash (36 %) y 490 B de RAM (24 %).

Acceder al código fuente del Problema E en GitHub

X. Gestión del código con GitHub

Los códigos del Laboratorio 4 se almacenan en el repositorio del curso para mantener trazabilidad de las modificaciones realizadas durante el desarrollo. Cada problema tiene su propia carpeta de código, junto con los diagramas de estados en draw.io y los archivos de simulación de Proteus.

Acceder al repositorio del Laboratorio 4 en GitHub

Las modificaciones significativas se registran mediante commits separados, de modo que pueda identificarse la evolución de cada problema. En el Problema B se destacan el ajuste del mapeo PWM al rango útil del motor, la integración de las bibliotecas TWI y LCD preservando las entradas analógicas y la protección por temperatura con histéresis. En el Problema E se registraron la versión inicial con sus correcciones, la integración de las bibliotecas de los docentes y la reorganización de la EEPROM mediante una estructura `EEMEM` con validación.

XI. Resultados generales

El Laboratorio 4 integró en cinco aplicaciones los periféricos del ATmega328P y distintos protocolos de comunicación. La Tabla XVIII resume el alcance logrado en cada problema.

Cuadro XVIII
Resumen de resultados por problema

Prob.	Interfaces principales	Estado
A	SPI, TWI, UART, EEPROM	Funcional en simulación y montaje
B	SPI entre dos MCU, ADC, PWM, TWI	Funcional en montaje
C	UART/Bluetooth, ADC, Timer0, TWI	Funcional en forma local; enlace Bluetooth no logrado
D	WS2812B, ADC, Timer2, EEPROM, PCINT	Funcional en simulación y montaje
E	TWI (MPU6050), WS2812B, Timer1, EEPROM	Compila; montaje no funcional

En el Problema A se verificaron en simulación y en el montaje la autenticación con tarjeta y PIN, el bloqueo tras tres intentos, las operaciones de retiro, depósito y consulta, y la persistencia de los saldos en EEPROM.

En el Problema B se comprobó el control distribuido mediante SPI: arranque y detención, regulación de velocidad con el PWM ajustado al rango útil del motor, detección y conteo de piezas con la LDR, accionamiento del servo y la protección por temperatura con histéresis.

En el Problema C se verificó de forma local el armado, el disparo en modo AUTO, el modo MANUAL, el silenciamiento y la visualización en la LCD, y la simulación confirmó la generación de las tramas por la UART. No se logró la comunicación entre el celular y el HC-05, por lo que el control remoto quedó sin verificar.

En el Problema D se verificaron la generación y reproducción de secuencias, el movimiento del cursor con zona muerta, los tonos por color, el fin de partida con guardado del récord en EEPROM y el reinicio por D4.

En el Problema E se desarrolló y compiló el firmware completo, pero el montaje físico no llegó a funcionar, por lo que su comportamiento quedó sin validación experimental.

XII. Conclusiones

El Laboratorio 4 permitió aplicar de forma integrada los principales periféricos del ATmega328P (ADC, temporizadores, PWM, UART, SPI, TWI, EEPROM e interrupciones) en cinco sistemas de naturaleza distinta, programados en lenguaje C. En todos los casos, organizar el funcionamiento mediante máquinas de estados facilitó el diseño, la depuración y la documentación, ya que cada sistema pudo analizarse estado por estado y las salidas quedaron definidas sin ambigüedad.

En el Problema A, la combinación de RFID por SPI, teclado matricial y LCD por I^2C resultó en un cajero funcional, en el que la EEPROM conservó los saldos entre reinicios y las validaciones de PIN, saldo y desborde protegieron al sistema frente a operaciones inválidas. Como limitación quedó una autenticación basada solo en el UID, sin validación criptográfica.

En el Problema B, la arquitectura maestro–esclavo por SPI separó con claridad la adquisición del accionamiento. Las pruebas físicas mostraron la importancia de ajustar el diseño al hardware real: el mapeo del PWM al rango 189–254 hizo útil el recorrido del potenciómetro, y una resistencia de valor incorrecto en el divisor de la LDR explicaba la falla inicial en la detección de piezas.

En el Problema C, canalizar los comandos remotos y los pulsadores por una misma función permitió validar toda la lógica de la alarma de forma local, y la base de tiempo de 1 ms con recepción por interrupción evitó retardos bloqueantes. Sin embargo, no se cumplió el objetivo de supervisión inalámbrica, porque no se logró el enlace entre el celular y el HC-05. Como la simulación confirmó que el microcontrolador genera las tramas, el problema se ubica en el enlace Bluetooth, que queda como punto pendiente.

En el Problema D, la generación de la señal de la matriz WS2812B mediante ensamblador de ciclos fijos resultó estable, y la calibración del centro del joystick al arrancar permitió usar el mismo programa en Proteus y en la placa real. El uso de retardos bloqueantes simplificó la implementación, a costa de demorar la atención del reinicio por D4.

En el Problema E se diseñó un procesamiento completo de la señal inercial (filtro complementario, media móvil exponencial, zona muerta y calibración persistente en EEPROM) y se resolvió una incompatibilidad de la biblioteca TWI con la lectura por *repeated start*. No obstante, el montaje físico

no llegó a funcionar, por lo que estas características quedan pendientes de verificación experimental.

En conjunto, el laboratorio mostró que la simulación es útil para validar la lógica y los protocolos, pero no reemplaza a las pruebas sobre el hardware real: la mayoría de los inconvenientes surgieron en el montaje (valores de componentes, conexiones, niveles de tensión y enlaces inalámbricos). Los resultados no logrados en los problemas C y E se documentan como trabajo pendiente para una próxima iteración.

REFERENCIAS

[1] UTEC, Ingeniería Mecatrónica, *Laboratorio 4 – Tecnologías de Microprocesamiento*, guía de laboratorio, Semestre IV, 2026.

[2] Microchip Technology Inc., *ATmega328P 8-bit AVR Microcontroller Datasheet*.

[3] AVR Libc Project, *AVR Libc Reference Manual* (módulos `<avr/eeprom.h>`, `<avr/interrupt.h>` y `<util/delay.h>`). [En línea]. Disponible: https://avrdudes.github.io/avr-libc/

[4] NXP Semiconductors, *MFRC522 Standard Performance MIFARE and NTAG Frontend – Product Data Sheet*.

[5] Texas Instruments, *PCF8574 Remote 8-Bit I/O Expander for I^2C Bus – Datasheet*.

[6] Hitachi, *HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver Datasheet*.

[7] STMicroelectronics, *L298 Dual Full-Bridge Driver Datasheet*.

[8] Aosong Electronics, *DHT11 Temperature and Humidity Sensor Datasheet*.

[9] Tower Pro, *SG90 Micro Servo Datasheet*.

[10] ITead Studio, *HC-05 Bluetooth to Serial Port Module Datasheet*.

[11] *HC-SR501 PIR Motion Detector Datasheet*.

[12] Worldsemi, *WS2812B Intelligent Control LED Integrated Light Source Datasheet*.

[13] InvenSense, *MPU-6000 and MPU-6050 Product Specification* y *Register Map and Descriptions*, Rev. 3.4.